

Final Presentation (LV-GridNav)

Language-Vision Guided Grid Navigation with Reinforcement Learning

PREVIOUS REVIEW

Project Title: 언어 및 시각 정보 기반 강화학습 알고리즘 구현

Problem Statement

인간은 언어로 목표를 이해하고 시각 정보로 현재 상태를 파악함으로써 종합적으로 판단을 한다.

이러한 학습 매커니즘은 환경과 끊임 없이 상호작용 하며 유연한 행동 정책을 학습할 수 있을 것으로 보여진다.

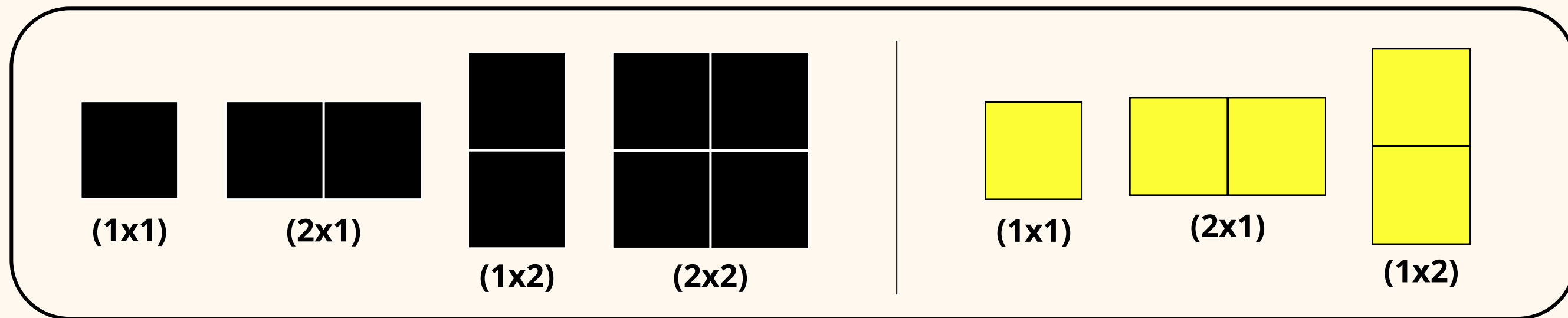
Project Purpose

- 언어와 시각 정보를 통합하여 이해하는 강화학습 Agent를 학습
- 2D 격자형 모의 환경에서 간단한 자연어 명령으로 지정된 목표 위치로 이동하는 Agent 학습
 - 언어 명령: 구체적인 목표 정의
 - CNN 모델: Agent 주변 환경 파악
 - 강화학습: 최적의 선택을 학습하는 알고리즘

Environment Setting

Default Configurations

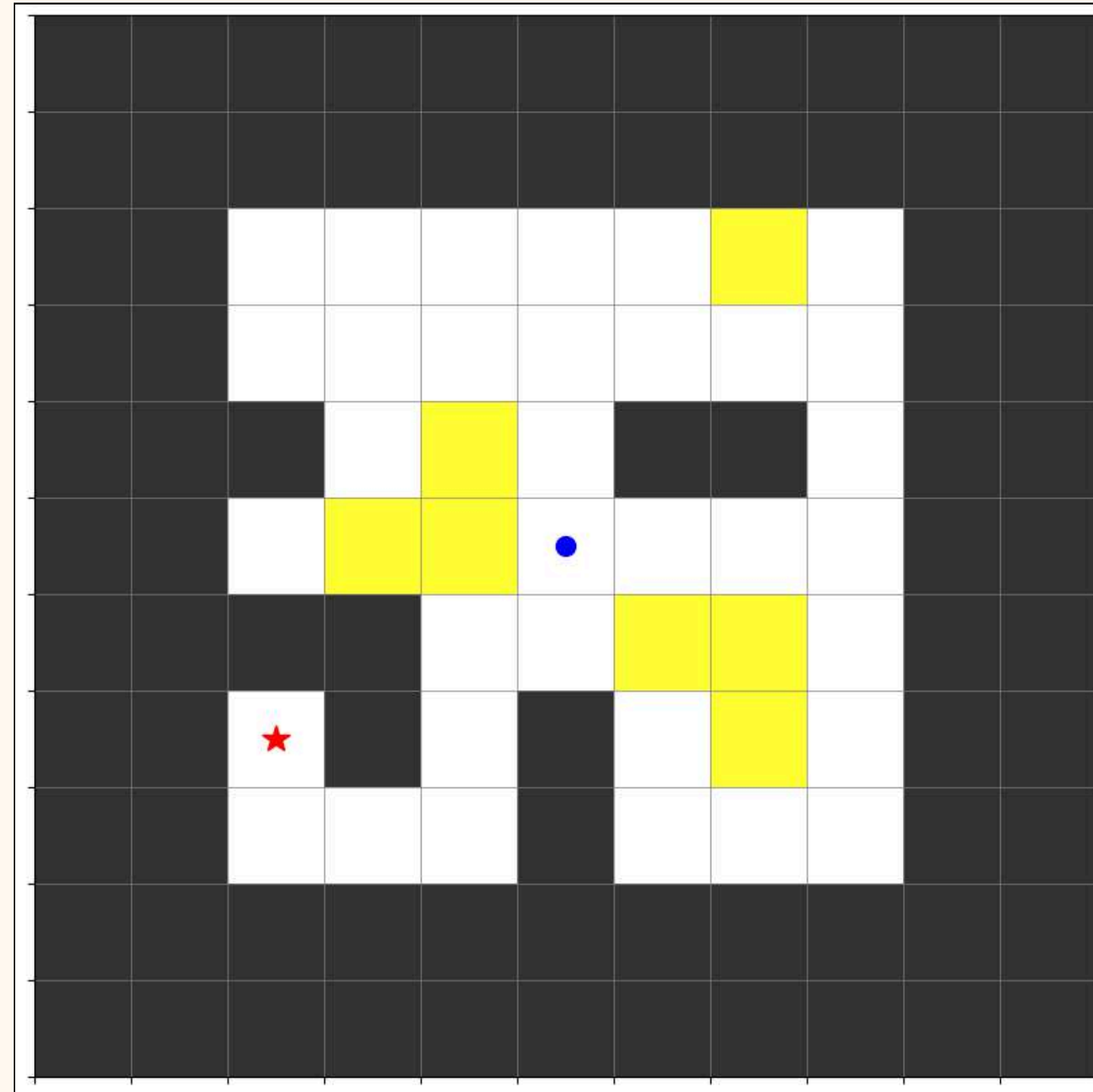
- **Grid Map Size:** 7x7 (with 2-cell Padding)
- **Goal Position:** Randomly Generated by Scenario (단, 서로 최소한의 거리를 유지)
- **Other Objects (Construction, Wall):** Randomly Generated with Pre-defined Offsets
 - *Construction* (공사 경로): 지나갈 수 있지만 일반 길에 비해 높은 패널티
 - *Wall* (벽): 지나갈 수 없는 길 (실제 학습 과정에서 지나갈 수는 있으나, 굉장히 높은 패널티)



▲ Offset Patterns for Construction (left) and Wall (right) Elements

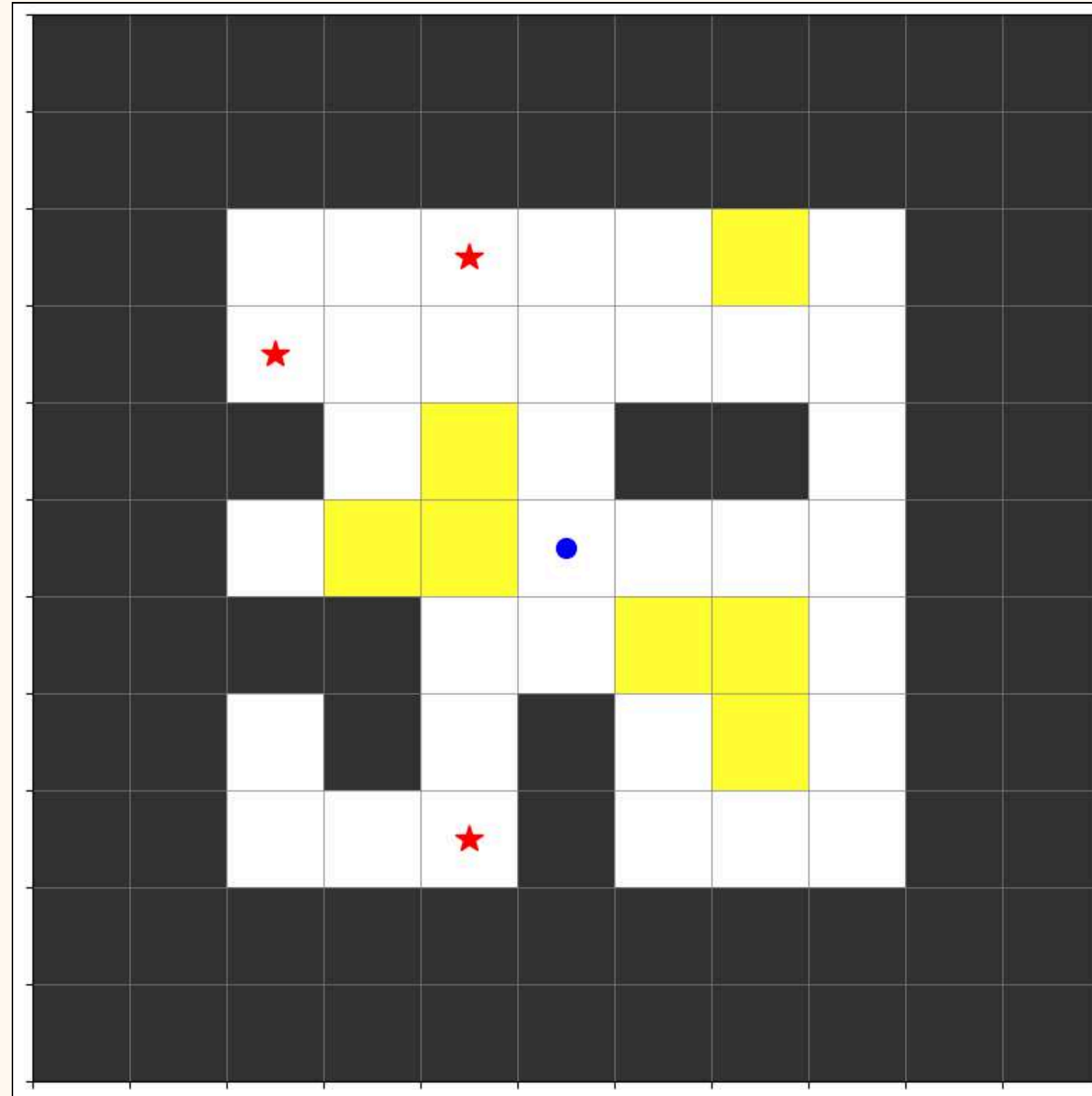
Scenario 1:

랜덤하게 정해진 단일 목표 지점으로 이동하기



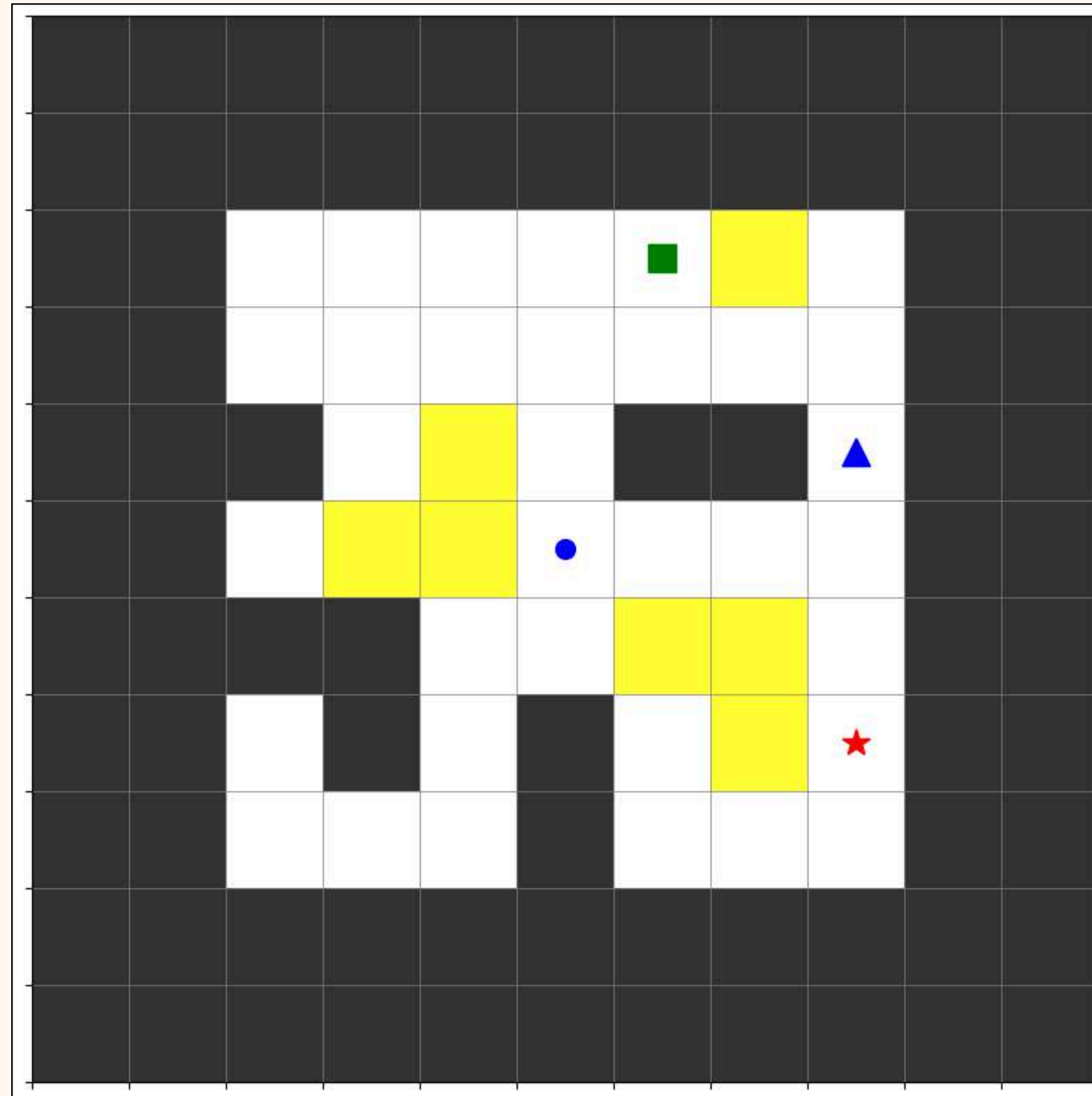
Scenario 2:

랜덤하게 정해진 여러 목표 지점 중 가장 가까운 지점으로 이동하기



Scenario 3:

랜덤하게 정해진 여러 목표 지점을 모두 방문하기

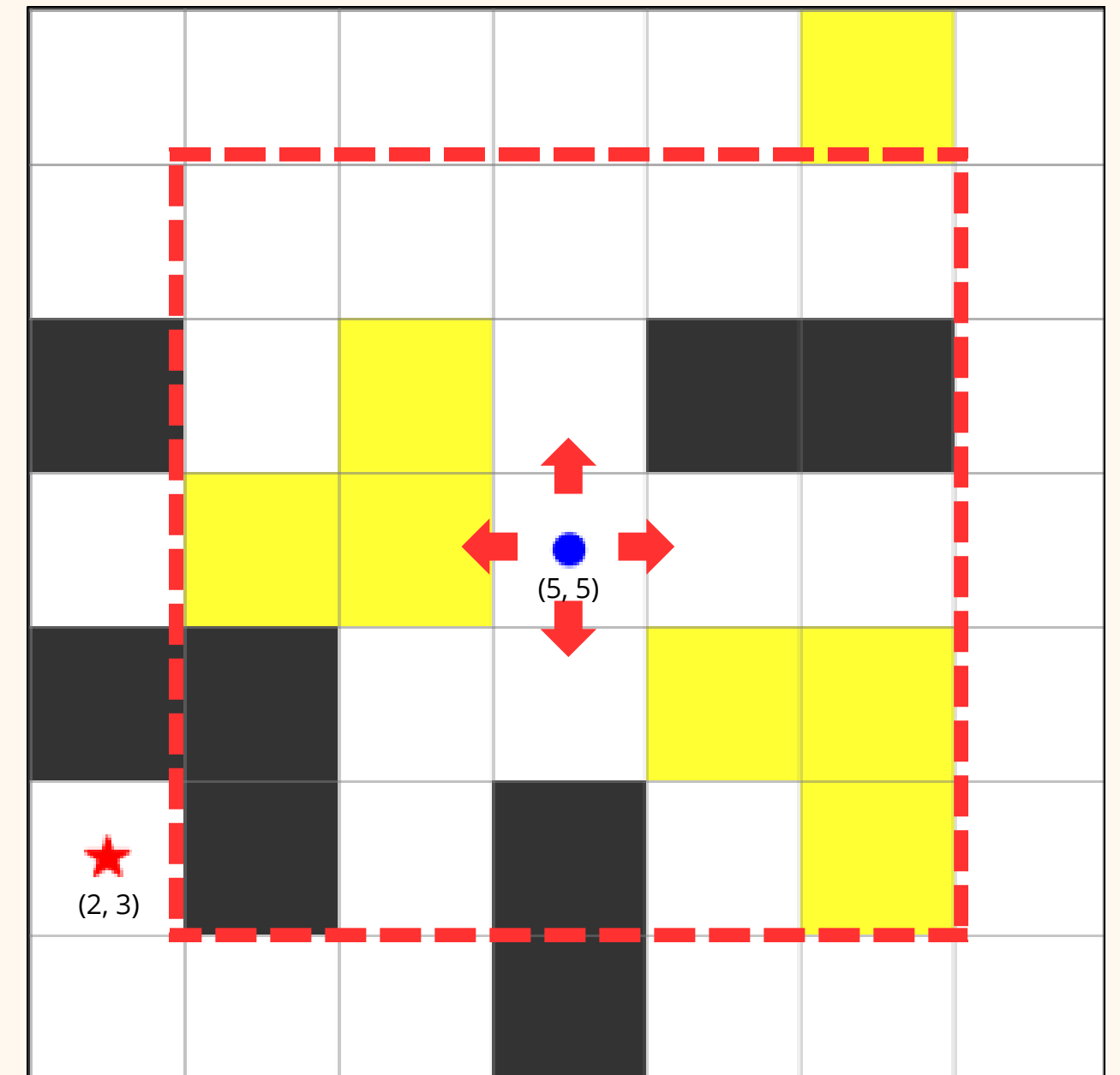


Agent Setting

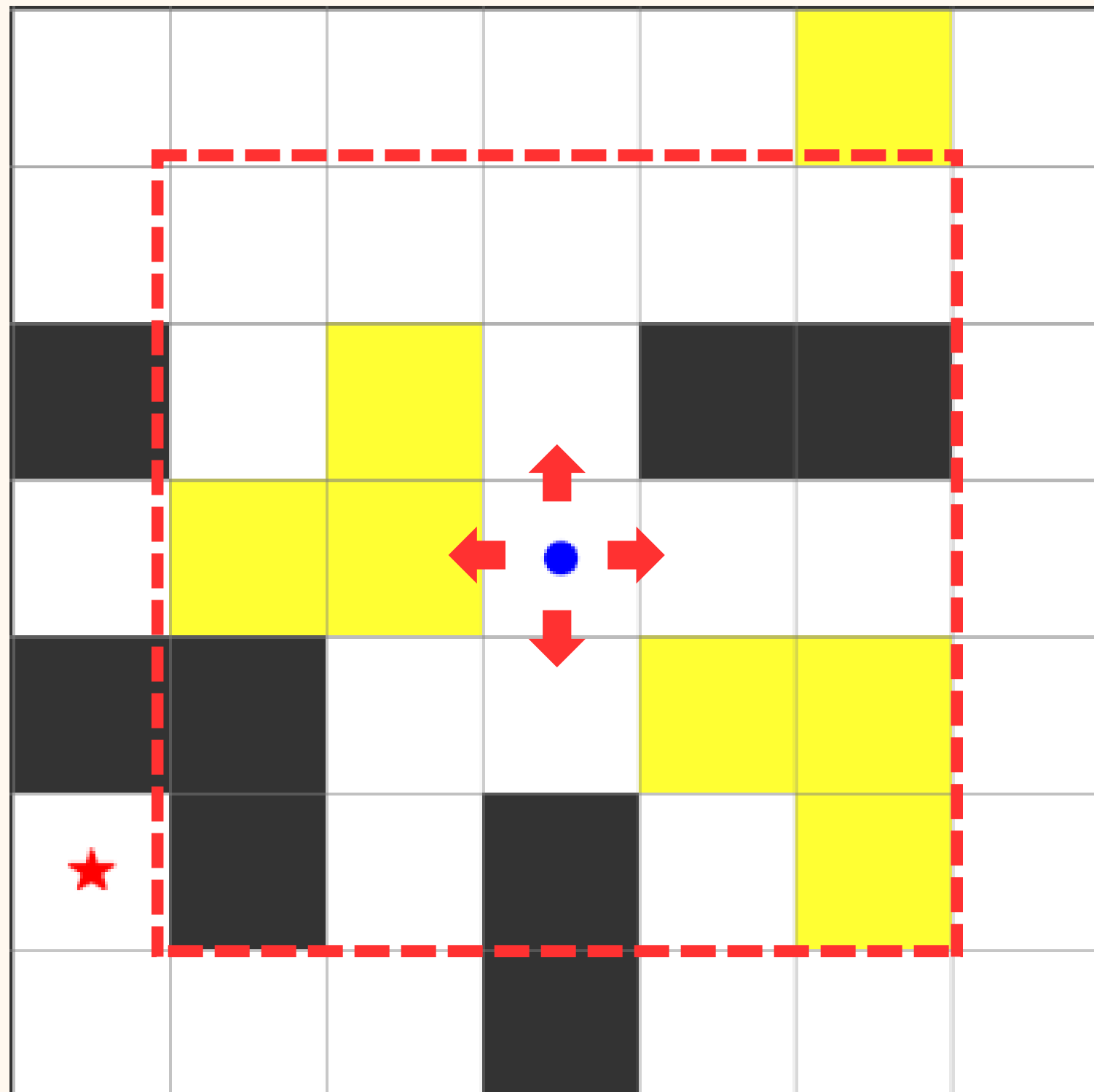
Detailed Specifications

- **Movement:** 4 Directions (Up / Down / Left / Right)
- **Step:** 1 cell per action
- **State:** 아래의 정보들을 조합한 29-dimensional vector
 - Current Position: **[x, y]**
 - Distance to Closest Goal: **[dx, dy]**
 - Surrounding Information (Vision Info)
 - **0: normal / 1: Construction / 2: Wall / 3: Goal**

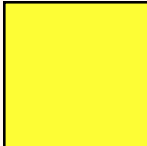
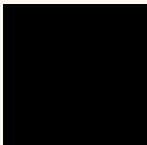
=> **[5, 5, -3, -2, 0, 0, 0, 0, 0, 0, 1, 0, 2, 2, 1, 1, 0, 0, 0, 2, 0, 0, 1, 1, 2, 0, 2, 0, 1]**



Reward System



Base Rewards

-  (normal) : **-0.1** [step penalty]
-  (construction) : **-0.3**
-  (wall) : **-5.0**
-  (goal) : **3.0**

Special Rewards

- 중복 방문 패널티: (방문 횟수) x (-0.2)
- 다중 목표 보상: (기본 목표 보상) x (1 + 0.5 x 방문 목표 수)
- 보상 Scaling: (각 step별 누적 보상) / 10.0

Methodologies

1. Policy Iteration

- 구현 배경:

- 강의에서 배운 순서에 따라 구현을 해보기 위해 초기에는 문제를 단순화하여 접근

- 알고리즘의 특징

- 전체 Map의 MDP를 모두 알고 있다는 가정 하에 동작
- 이로 인해 앞서 정의하였던 세부 State 정보는 무시하고, 단순히 Value만을 기반으로 정책을 비교

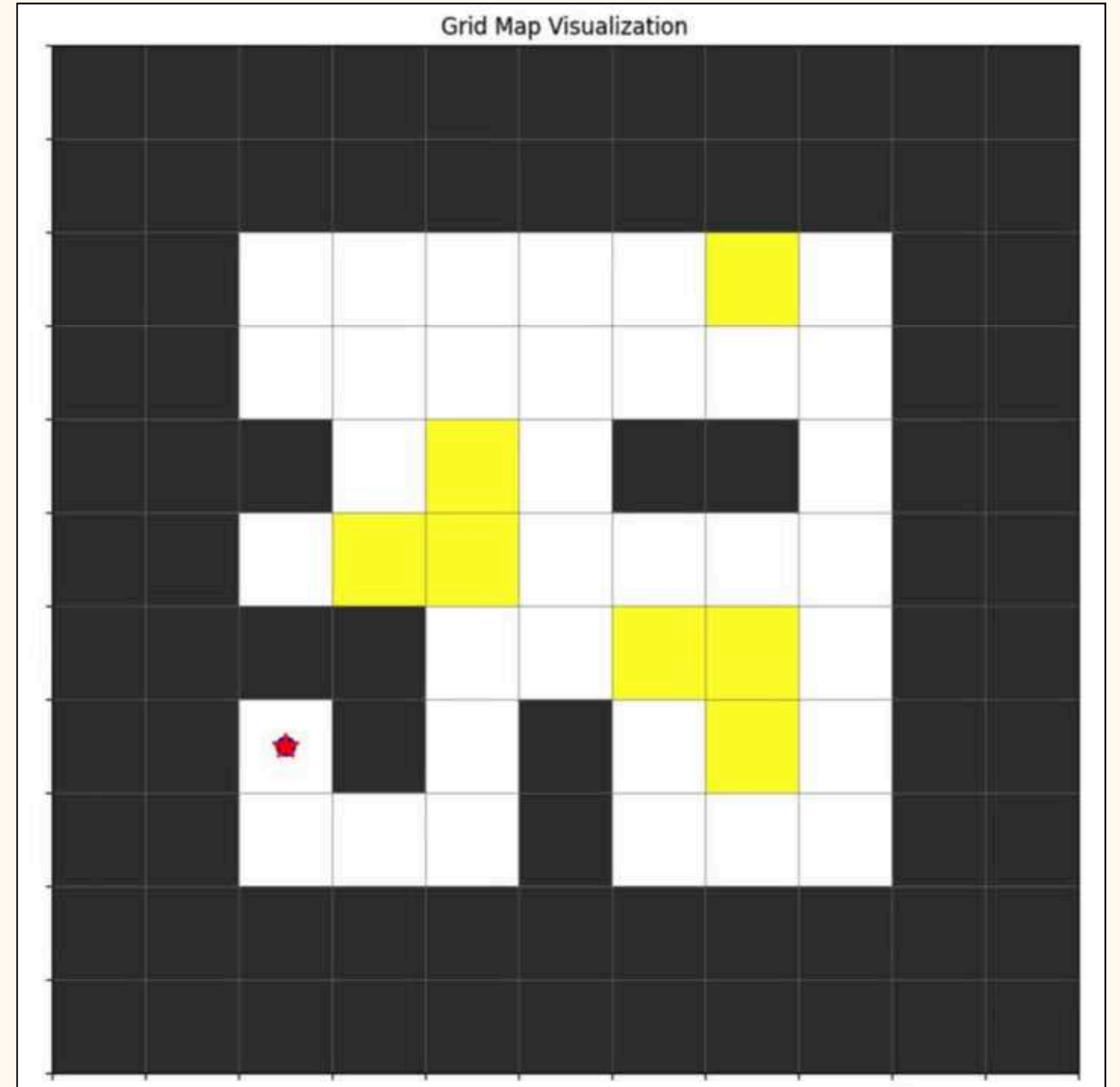
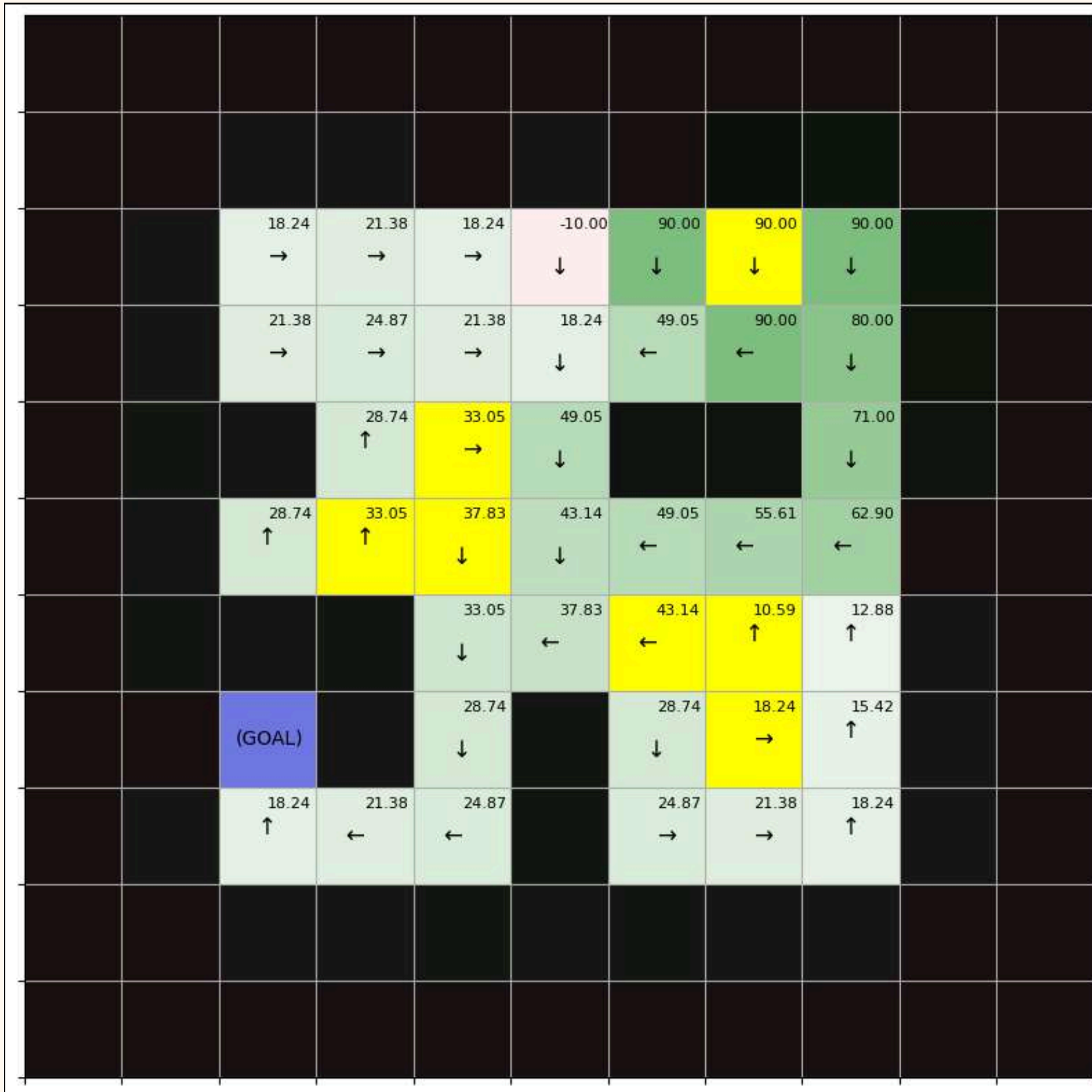
- 결과

- Scenario1 / Scenario2 에서는 **매우 빠르게 수렴하며 우수한 성능**을 보임
- Scenario3은 **어떻게 접근하더라도 해결하지 못했다.**

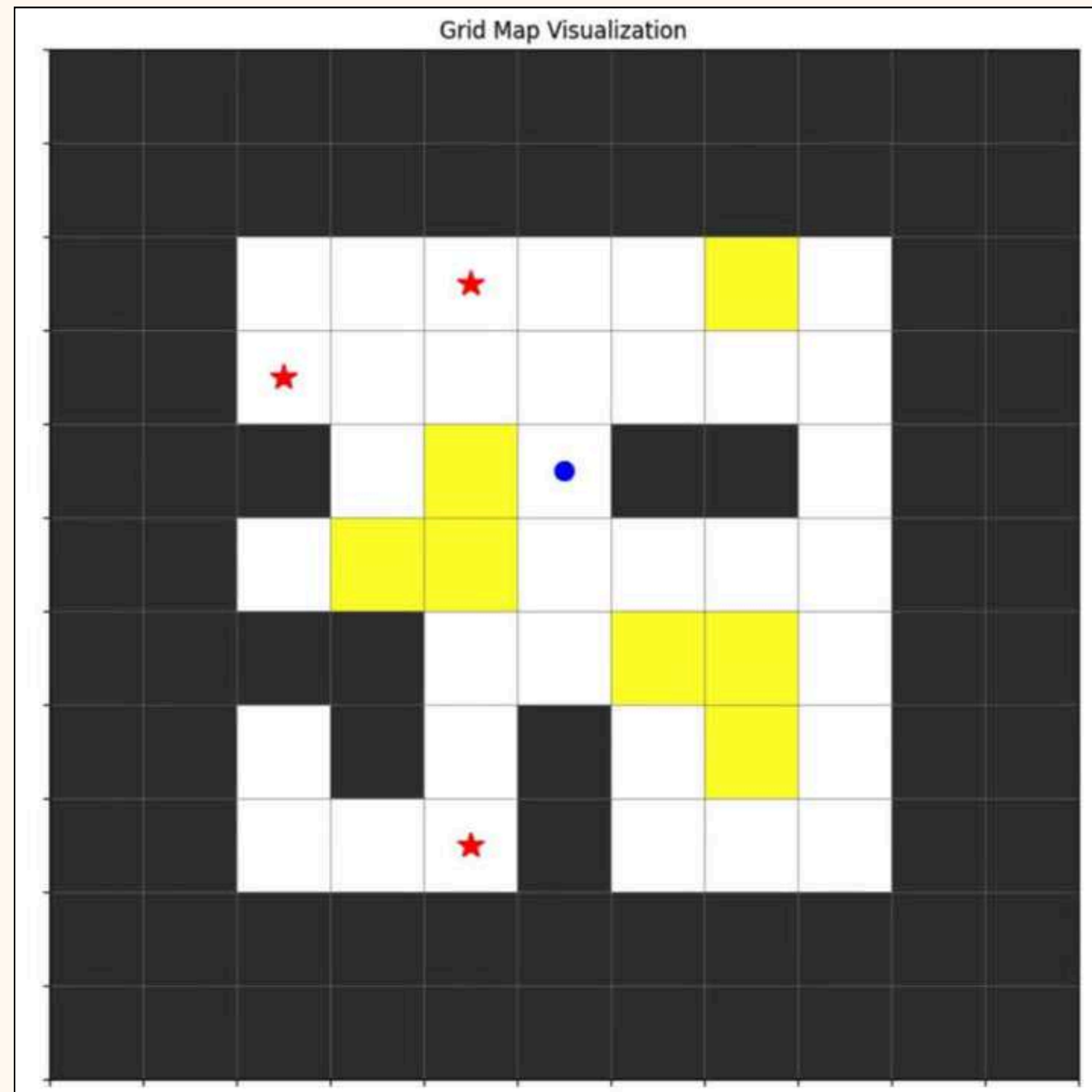
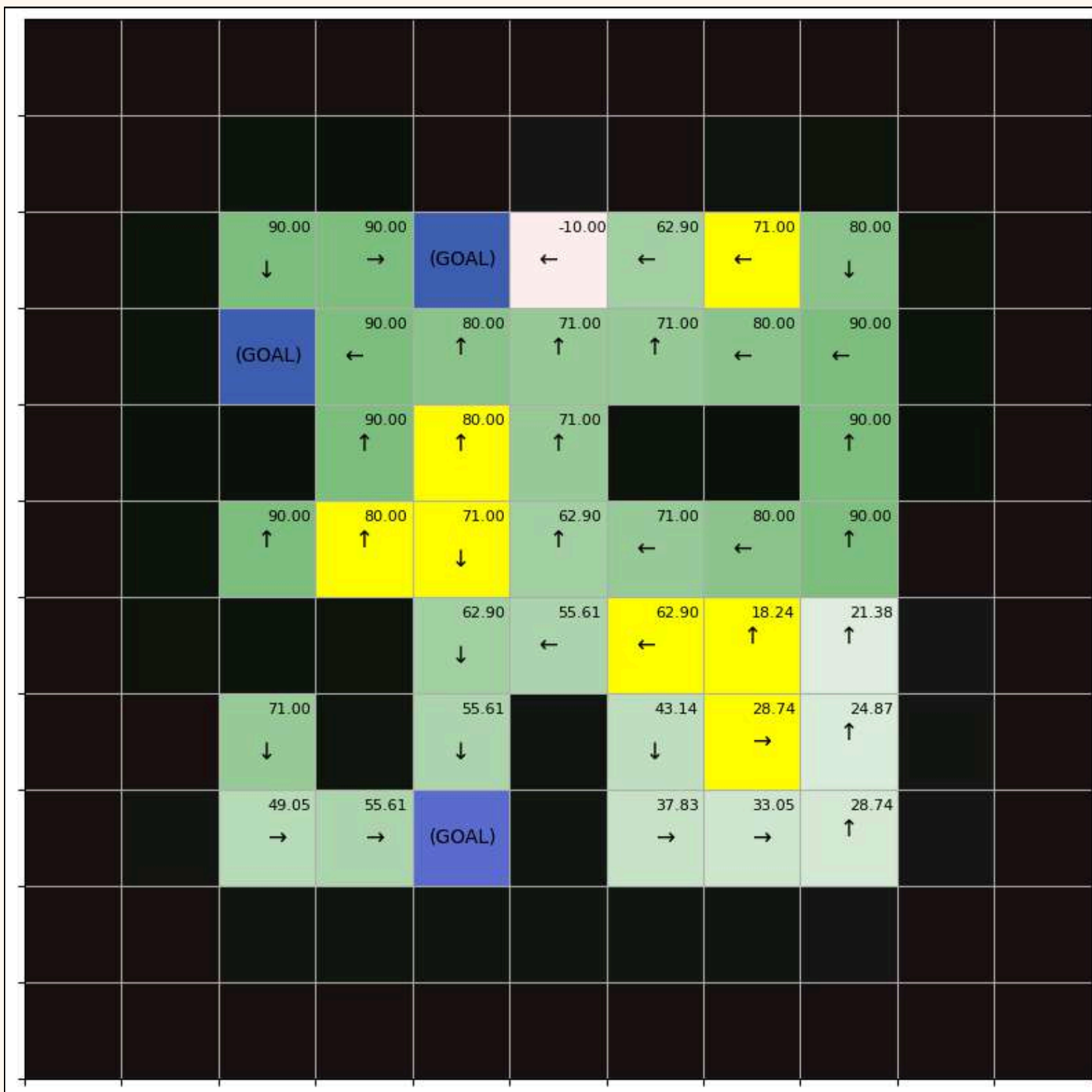
정책 반복 알고리즘 학습 시작...
정책이 3번째 반복에서 수렴했습니다.
학습 완료! 소요 시간: 0.38초

- 정책 반복법은 각 타일에서 얻어지는 보상을 기반으로 모든 정책의 가치를 미리 계산하여 최적의 정책을 결정한다. 따라서 동적으로 변화하는 보상(방문한 목표 지점의 가치 소멸)이나 변화하는 목표 지점과 같이 즉각적인 대응을 요구하는 상황은 고려하기 어렵다.

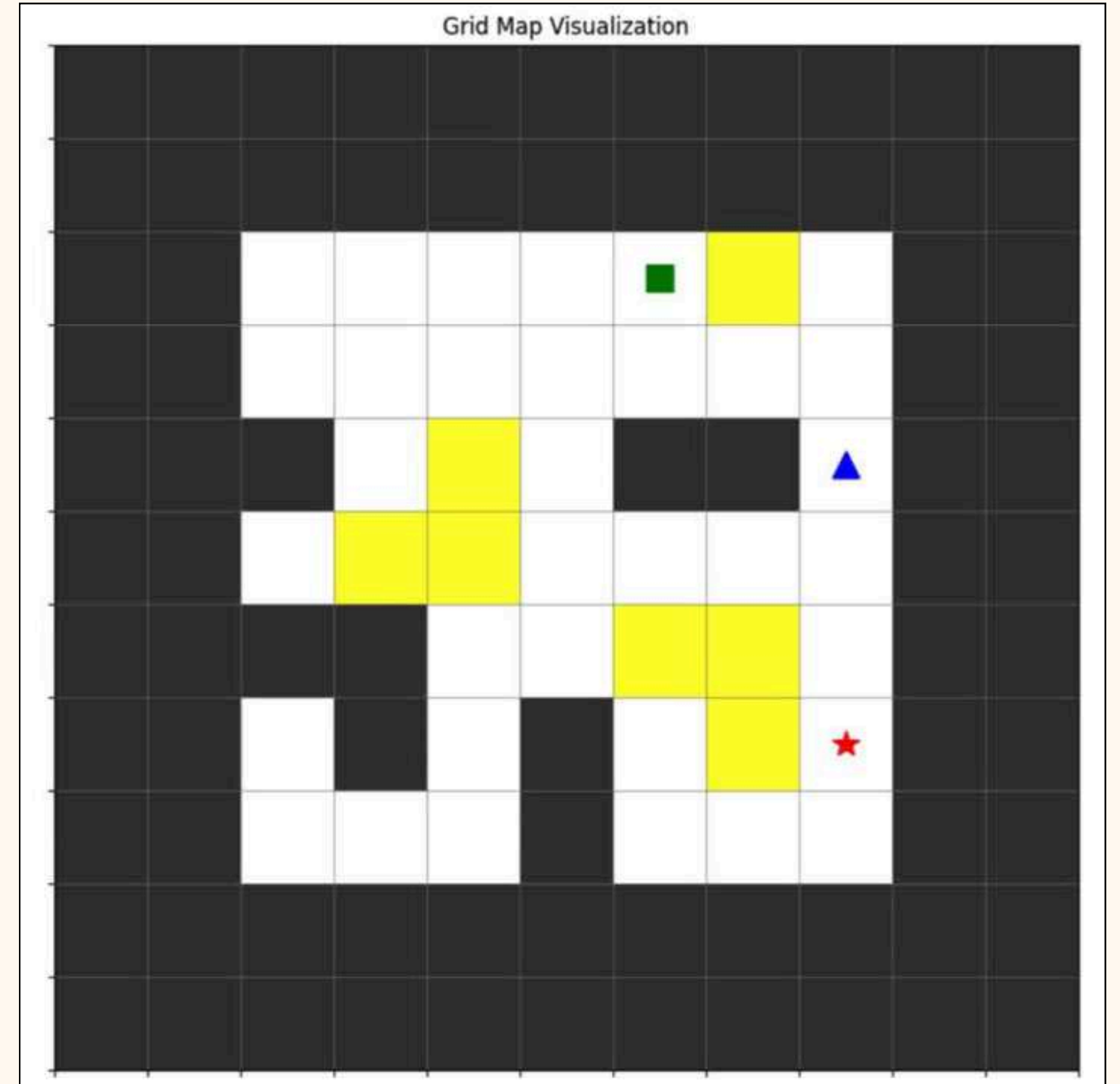
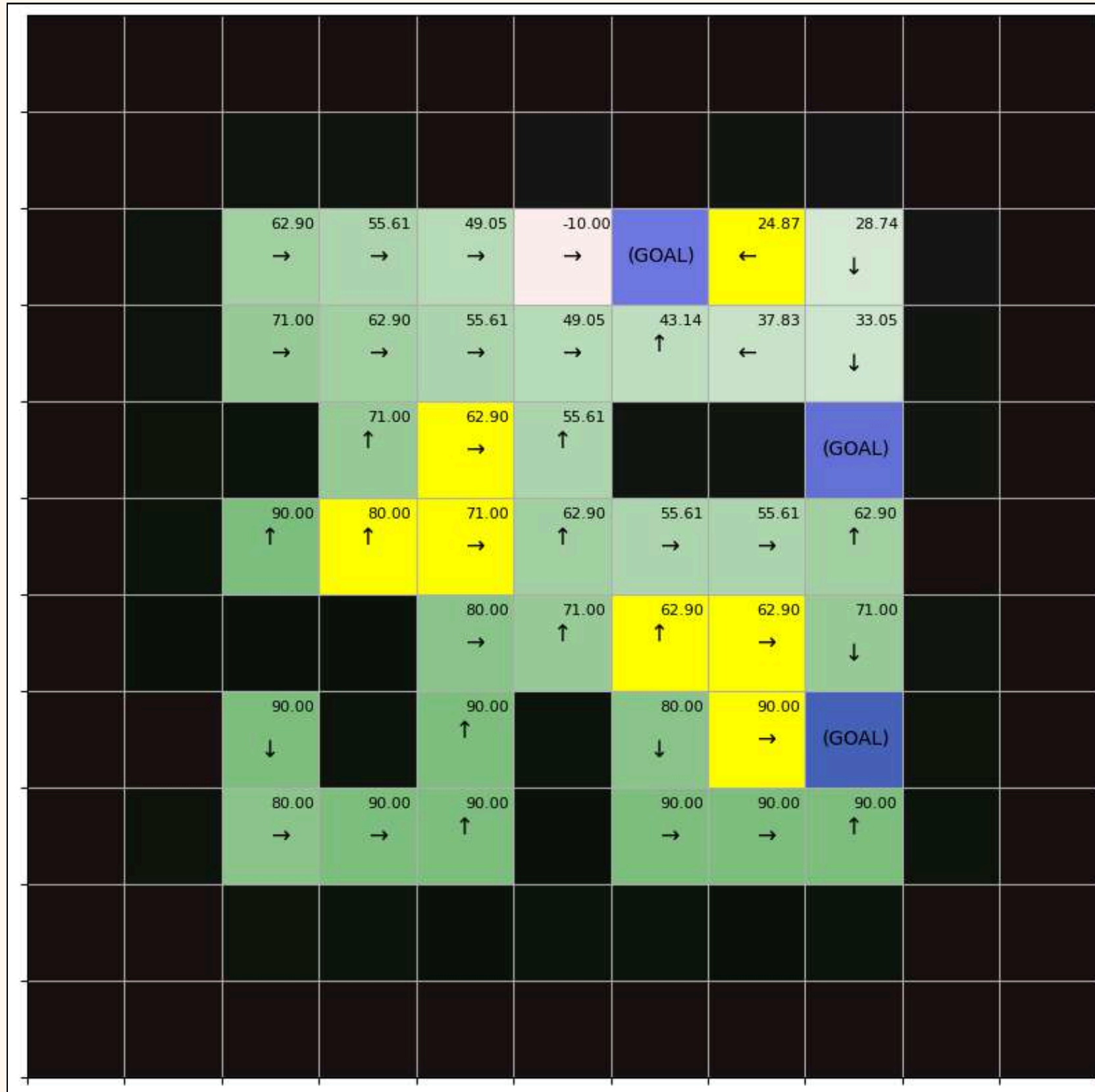
< Policy Iteration: Scenario1 >



< Policy Iteration: Scenario2 >



< Policy Iteration: Scenario3 >



Methodologies

2. Q-Learning

- 구현 배경:

- “Scenario3”을 해결하기 위해서는 agent의 실시간 적응 능력이 필요
- 초기 구현 목표에 따라 전체 맵 정보는 모른 채 오직 주변 정보와 경험을 활용하여 행동을 결정하도록 문제를 고도화

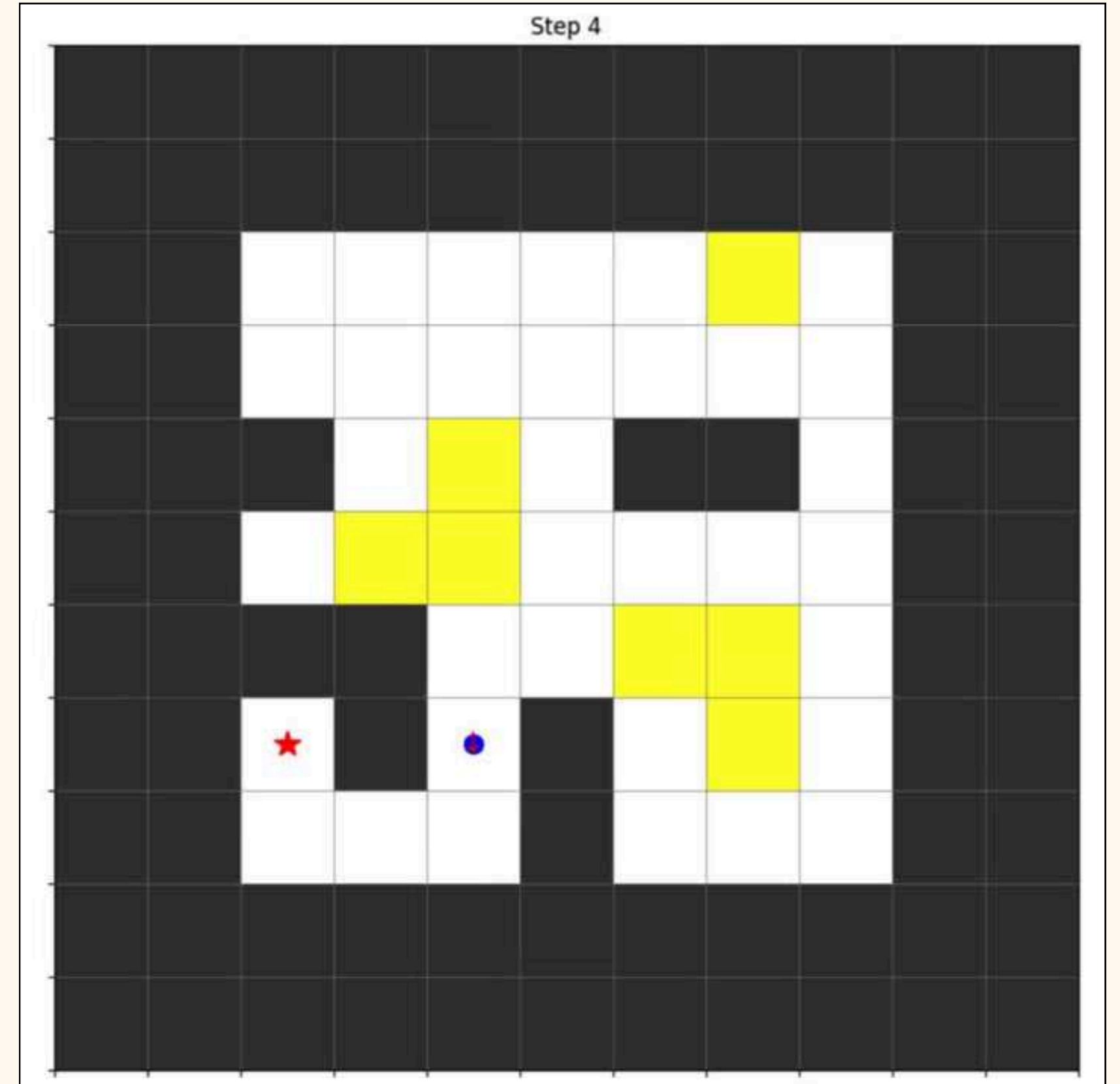
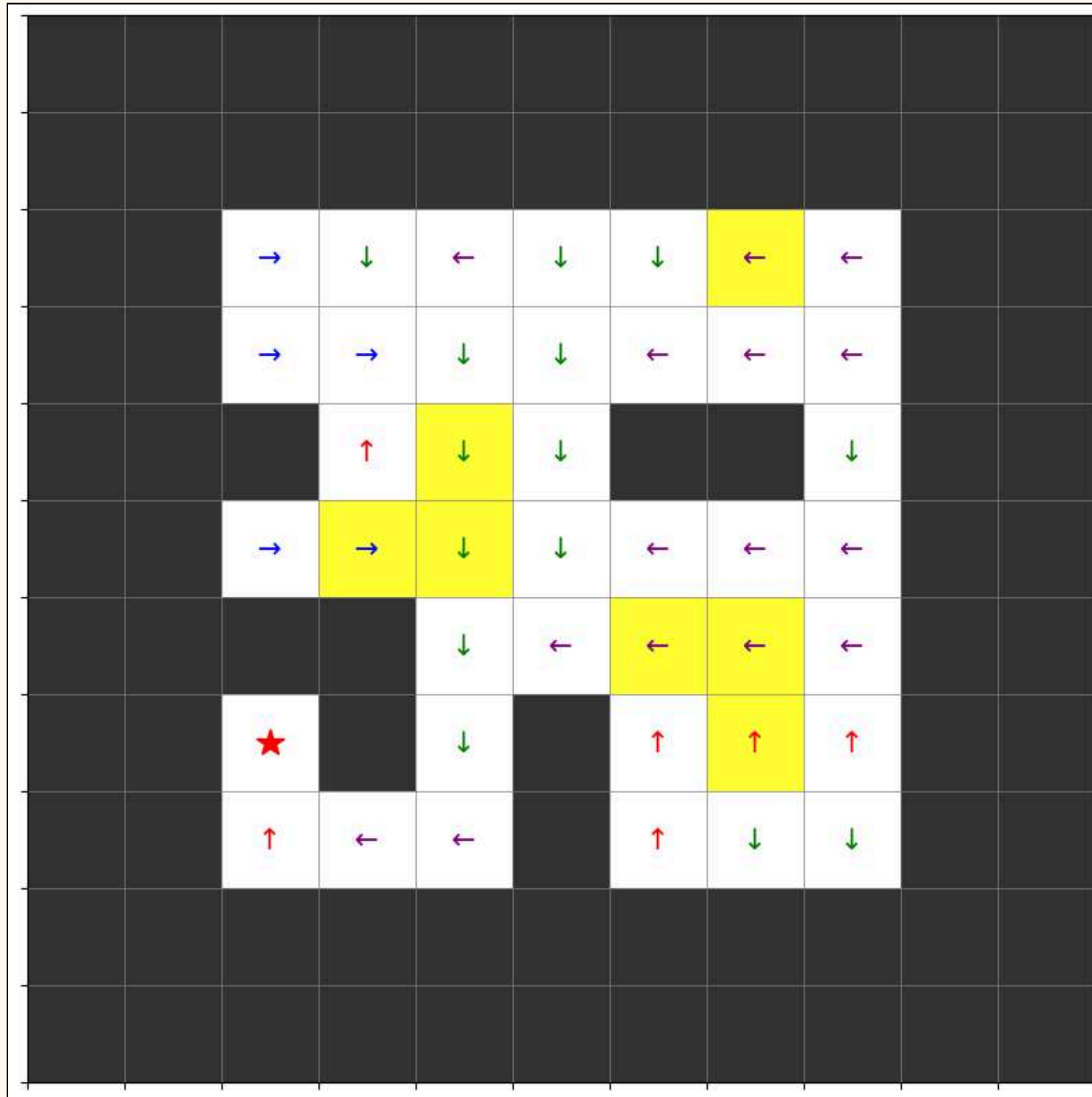
- 알고리즘의 특징

- Agent가 환경을 직접 탐험하며 얻는 보상(Q-value)을 바탕으로 정책을 실시간으로 업데이트

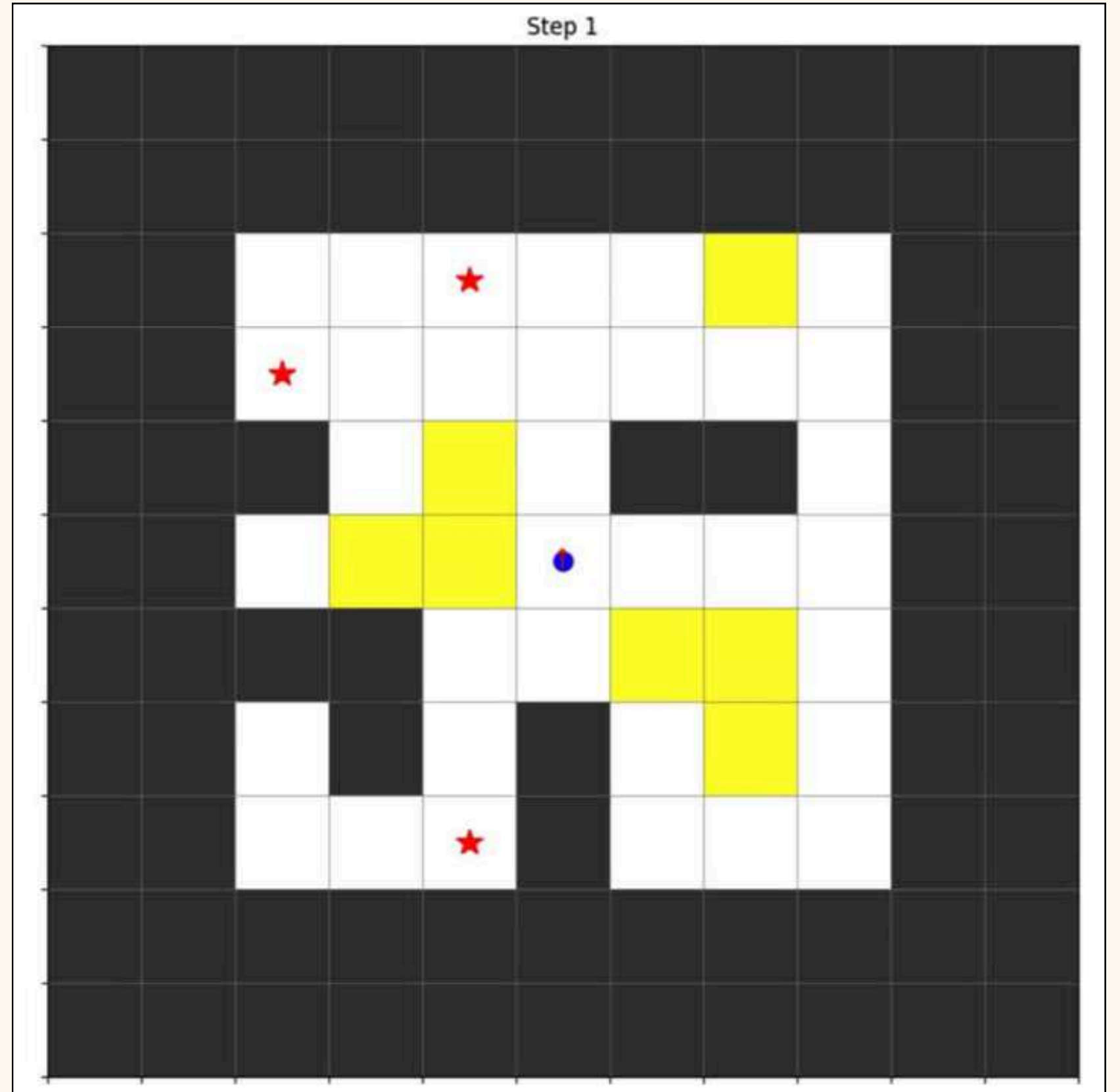
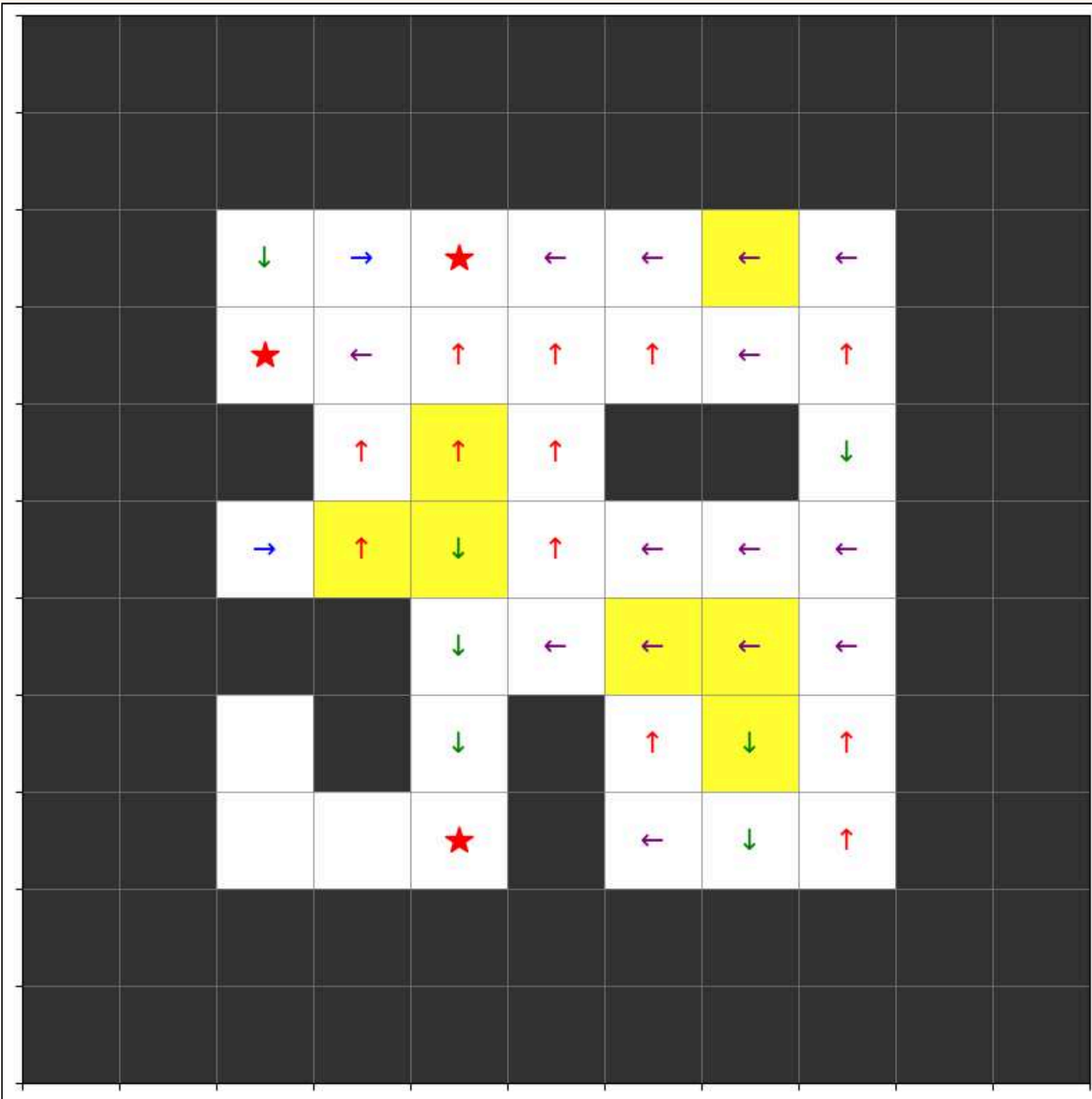
- 결과

- Scenario3을 포함하여 **모든 목표를 성공적으로 수행**
- **비효율적인 경로**: 목표를 달성하긴 하지만, 최적의 경로라고 보기는 어려움
- **학습 불안정성**: 학습을 시도할 때마다 결과가 크게 달라지며, 목표를 달성하지 못하는 경우도 많음
- **긴 학습 시간**: Scenario3을 해결하기 위해 40분 이상의 학습 시간이 소요

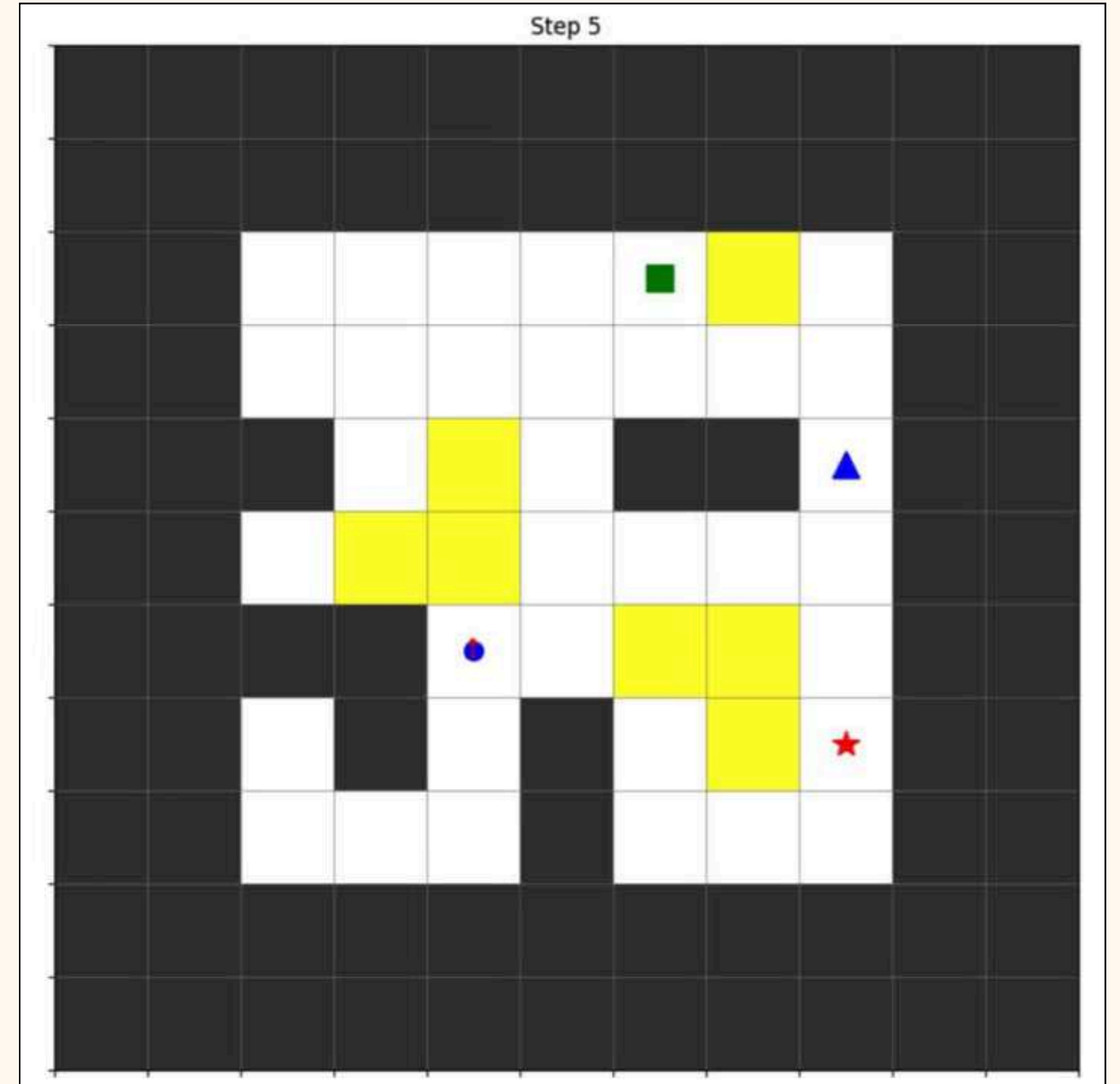
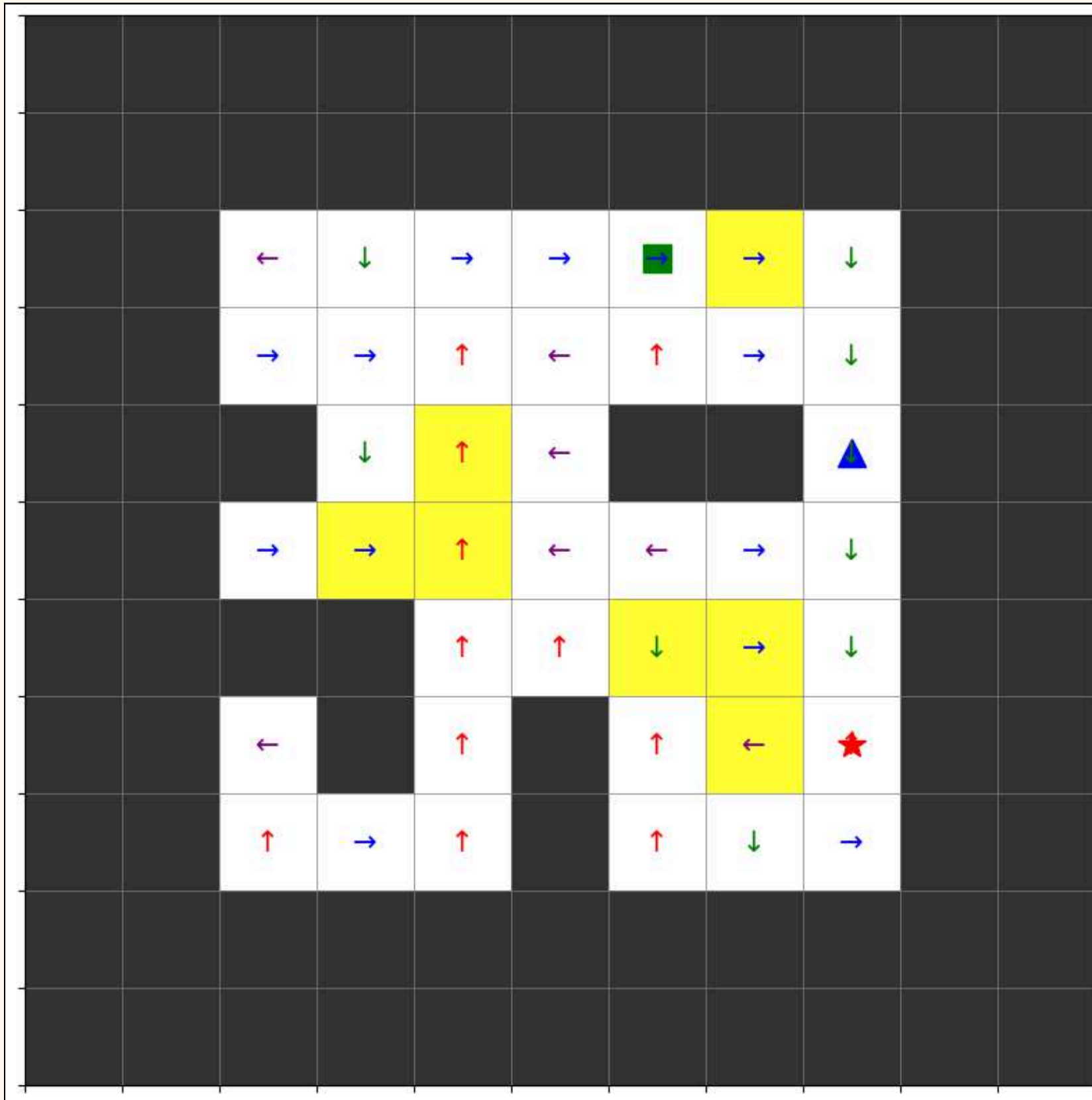
< Q-Learning: Scenario1 >



< Q-Learning: Scenario2 >



< Q-Learning: Scenario3 >



Methodologies

3. DQN (Deep Q-Network)

- 구현 배경:

- Agent가 주변 정보(State)를 효과적으로 학습하여 인지하고 복잡한 환경에 잘 적응할 필요성

- 알고리즘의 특징

- Q-Learning의 "Q-table"을 "심층신경망(Deep Neural Network)"으로 대체하여, 방대한 상태 공간에 대한 Q-value를 효과적으로 추정

- 결과

- Policy Iteration과 Q-Learning으로는 아무리 해도 해결하기 어려웠던 "**Scenario3**"를 **빠르고 안정적으로 해결**했습니다.
 - (Q-Learning: 약 45분) → (DQN: 약 3분)
- 비효율적으로 움직이던 Q-Learning과 달리, DQN은 **가장 효율적인 최적 경로를 찾아 목표를 완수**
- 여러 번의 학습 시도에도 **안정적이고 일관된 결과**를 보임

<DQN Agent>

```
class DQNAgent:
    def __init__(self, state_size, action_size):
        self.gamma = 0.99
        self.lr = 0.001
        self.epsilon = 1.0          # 초기 epsilon 값
        self.epsilon_min = 0.01     # 최소 epsilon 값
        self.epsilon_decay = 0.992  # epsilon 감소 속도
        self.buffer_size = 20000    # 경험 재생 버퍼 크기
        self.batch_size = 64        # 미니배치 크기
        self.action_size = action_size
        self.state_size = state_size

        # 경험 재생 버퍼
        self.replay_buffer = ReplayBuffer(self.buffer_size, self.batch_size)

        # Q-Network
        self.qnet = QNet(self.state_size, self.action_size)
        self.qnet_target = QNet(self.state_size, self.action_size)
        self.optimizer = optim.Adam(self.qnet.parameters(), lr=self.lr)
        self.loss_fn = nn.MSELoss()
```

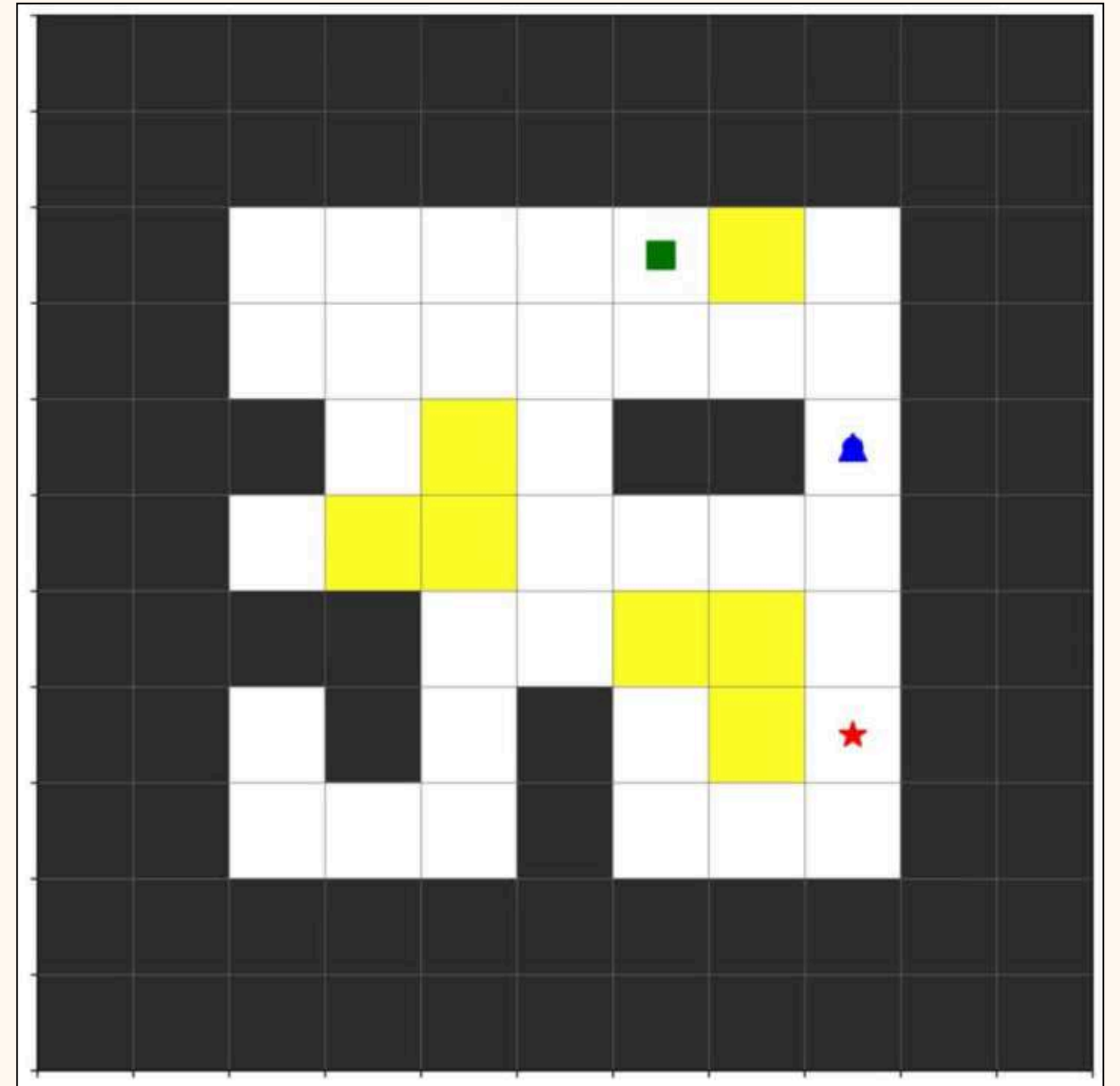
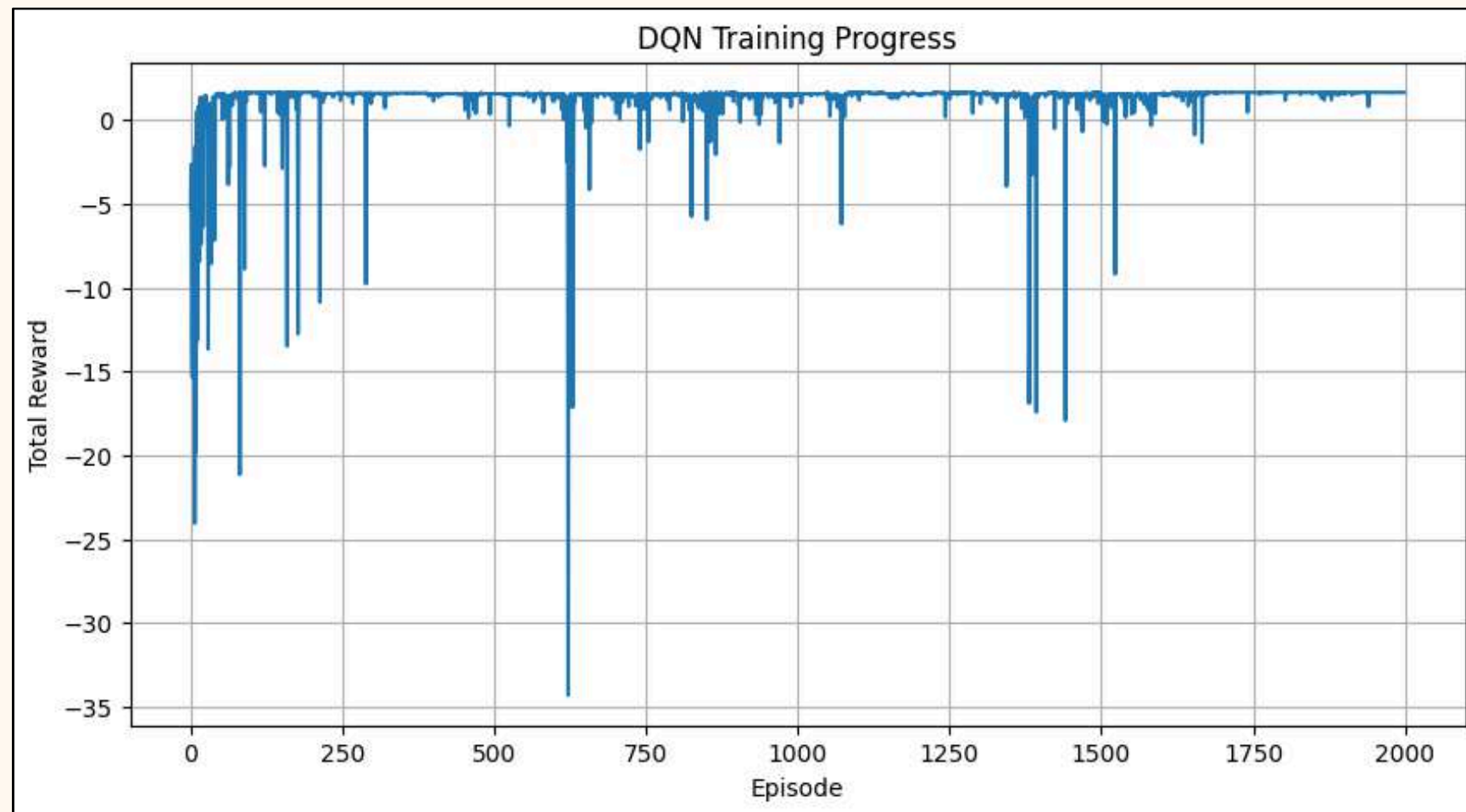
<Q-Network>

```
class QNet(nn.Module):
    def __init__(self, state_size, action_size):
        super().__init__()
        self.l1 = nn.Linear(state_size, 128)
        self.l2 = nn.Linear(128, 256)
        self.l3 = nn.Linear(256, 128)
        self.l4 = nn.Linear(128, action_size)

        # Dropout 추가
        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        x = torch.relu(self.l1(x))
        x = self.dropout(x)
        x = torch.relu(self.l2(x))
        x = self.dropout(x)
        x = torch.relu(self.l3(x))
        x = self.l4(x)
        return x
```

< DQN: Scenario3 >



Limitatinos

(1) 모든 시나리오에 강건한 Agent 학습 실패

- **프로젝트 초기 목표:** 랜덤하게 생성되는 맵과 목표 지점에서도 잘 동작하는 하나의 범용적인 Agent 학습
- **시도 및 한계:** 매 episode마다 시나리오를 다르게 생성하도록 학습을 시도했으나, 불안정한 환경 변화 (특히 잦은 목표 지점 변경)로 인해 학습이 성공적으로 이루어지지 못함
 - **원인 분석:** 잦은 환경 변화가 강화 학습 환경의 불안정성을 가중시키고, Target Network의 정상적인 동작을 방해한 것으로 판단
- **해결 아이디어:** 시나리오 변경 주기를 조절하고, Target Network 동기화 방식 및 복잡한 시나리오에 대한 가중치 학습을 고려한다면 일반화된 Agent를 학습할 수 있을 것으로 기대

Limitatinos

(2) 더 큰 Scale에서의 성능 한계

- **예상과 다른 결과:** 현재 구현했던 방식에서 맵 크기를 확장($7 \times 7 \rightarrow 21 \times 21$)했을 때, 예상과는 달리 기대만큼의 성능을 보이지 못했다.
- **문제 해결 가능성 확인:** 몇 가지 실험을 거쳐 65% 수준의 성공률을 달성.
- **개선 가능성:** 다양한 상태 정의, 복잡한 모델 사용, 체계적인 보상 함수 설계 등을 통해 넓은 환경에서도 잘 동작하는 Agent를 학습 시킬 수 있을 것으로 기대

(3) 초기 구현 목표와의 Gap

- **초기 목표:** 언어 및 시각 정보의 feature를 딥러닝 모델로 벡터화하여 Agent의 State로 활용
- **실제 구현:** 여러 가지 제약(시간/지식의 한계)로 인하여 매우 단순화된 형태의 시스템으로 대체
 - **언어 정보:** 시나리오 기반의 목표 생성으로 간소화
 - **시각 정보:** 타일 특성 기반의 단순 encoding vector 형태
- **긍정적 평가:** 비록 초기 목표를 완벽히 구현하진 못했지만, 현재 풀고자 했던 문제의 복잡도를 고려할 때 오히려 현재와 같이 단순화된 접근 방식이 더 효율적인 해결책이었을 수 있겠다는 생각

Comments

1. 강화학습과 Agent의 무궁무진한 잠재력

- 강화학습은 철저한 환경 세팅, 반복 가능한 시뮬레이션, 체계적인 보상 함수 설계 등 까다로운 제약 조건 많다.
- 단, 이러한 조건들만 충족된다면, agent를 어떻게 설계하느냐에 따라 활용 가능한 잠재력이 무궁무진하다는 점을 느꼈다.

2. 학습 환경 세팅 정말정말 중요하다

- “환경 세팅과 보상 함수 설계가 사실상 강화학습의 전부다”라는 말에 깊이 공감하였다.
- 마치 버그가 전혀 없는 완벽한 게임을 만드는 것과 같이 치밀하게 환경과 agent의 행동을 정의해야만 agent가 올바르게 학습한다는 것을 직접 경험했다.
 - 결국 agent가 주어진 환경에서 학습을 하기 때문에, 이상한 환경은 잘못된 학습으로 이어지는 것은 어찌 생각해보면 당연한 일이기도 한 것 같다.

3. 보상 함수 설계 정말정말정말 중요하다

- 보상 체계를 어떻게 설계하느냐에 따라 agent의 학습 능력에 엄청난 차이가 발생한다는 사실을 경험했다.
 - 초기에는 직관적으로 보상을 설계하였으나, 이는 불안정한 학습의 주된 원인이었다. 보상 체계가 논리적인 맥락 하에 설계되고, 누적 보상이 적절한 범위(e.g. -10 ~ 10)를 유지할 때 모델이 안정적으로 수렴함을 알게 되었다.
- 하지만 이런 중요성에도 불구하고, 어떤 기준으로 보상 함수를 설계해야 하는 지에 대한 명확한 기준은 여전히 모르겠다.

THANK YOU

Project 깃허브 주소: https://github.com/editdiary/RL-project_LV-GridNav